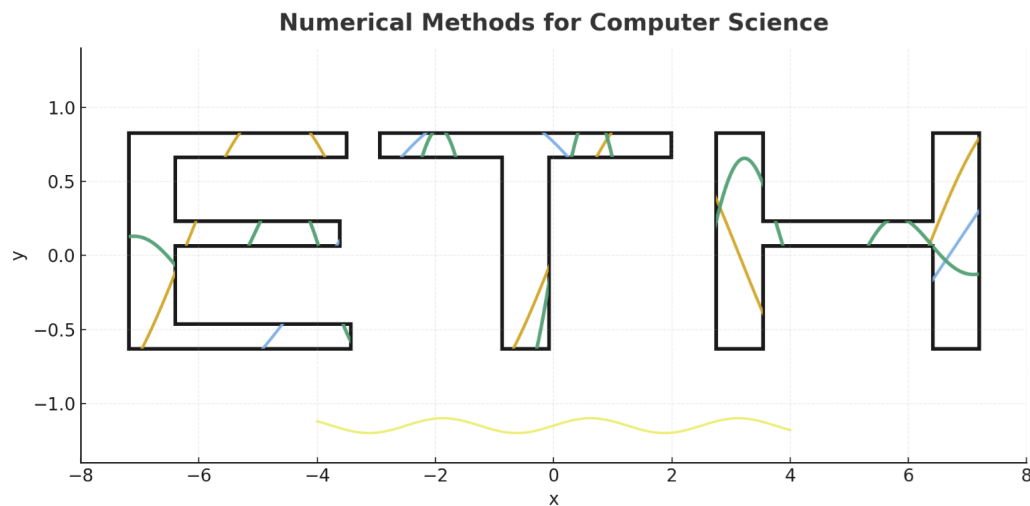




Numerical Integration and Quadrature Methods — Week #6

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This week introduces numerical integration, or *quadrature*, the art of approximating integrals with finite weighted sums. We explore the Trapezoidal and Simpson rules, composite formulas, the Romberg method, and Gaussian quadrature, bridging intuition, precision, and computational efficiency.

Contents

1	Motivation and Overview	1
1.1	Everyday intuition	1
1.2	Why do we need this in Computer Science?	2
1.3	Analytic vs. numeric integration	3
1.4	Example: measuring the area under a curve	3
2	Fundamental Concepts	4
2.1	General idea	4
2.2	Error and convergence	4
2.3	Polynomial interpolation and the Lagrange form	5
2.4	Accuracy and polynomial degree	7
2.5	Divide and conquer: composite formulas	7
3	Formulas with Equally Spaced Nodes	8
3.1	Midpoint Rule and its Derivation from Lagrange Interpolation	8
3.2	Trapezoidal Rule and its Derivation from Lagrange Interpolation	10
3.3	Simpson's Rule and its Derivation from Lagrange Interpolation	11
3.4	Composite Formulas	13
4	Bonus: Monte Carlo Integration	16
5	Numerical Integration in Python	18
5.1	Setup	18
5.2	Manual Implementations	18
5.3	Evaluate and Compare	19
5.4	Using <code>scipy.integrate</code>	19
5.5	Typical Output Example	20

1 Motivation and Overview

In mathematics and physics, integrals are used to measure quantities that accumulate continuously, like areas under curves, distances from velocity, probabilities from density functions, and much more.

But in computer science, we rarely know an exact *formula* for the integral. Instead, we often know the function only at discrete points — for example, as data from a simulation, an image intensity curve, or a sensor signal. So how can we compute the “area under the curve” from just a few samples?

That is exactly what **numerical integration**, or *quadrature*, is about.

Note

Goal: approximate an integral

$$I = \int_a^b f(x) dx$$

by a finite sum

$$Q_n(f) = \sum_{i=1}^n w_i f(c_i)$$

where c_i are chosen sample points (called *nodes*) and w_i are numerical weights.

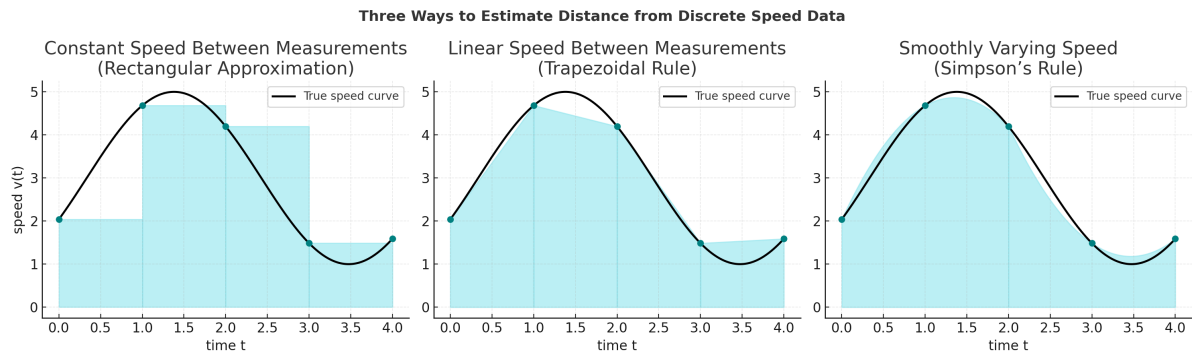
The idea is simple but powerful: instead of computing the exact area, we approximate the curve by a shape whose area we *can* compute: a rectangle, a trapezoid, or a parabola and sum up the results.

1.1 Everyday intuition

Think about estimating the distance a car has traveled when you only know its speed every few seconds. Depending on how you assume the speed changed between those points, you obtain very different approximations of the total distance — that is, of the area under the speed curve.

- If you assume the speed is **constant** between measurements, you’re using a simple **rectangular rule**.
- If you assume the speed changes **linearly**, you’re using the **Trapezoidal rule**.
- If you assume the speed changes **smoothly**, following a small parabola between consecutive points, you’re using the **Simpson rule**.

In all cases, we replace the possibly complicated function $v(t)$ by a simpler one whose “area” (i.e., the total distance) is easy to compute.



Three ways to estimate distance from discrete speed data: (left) constant speed approximation, (middle) linear change (Trapezoidal Rule), and (right) parabolic change (Simpson's Rule).

Each method corresponds to a different mathematical model of the function $v(t)$:

$$\text{Constant: } f(x) \approx c, \quad \text{Linear: } f(x) \approx a + bx, \quad \text{Quadratic: } f(x) \approx a + bx + cx^2.$$

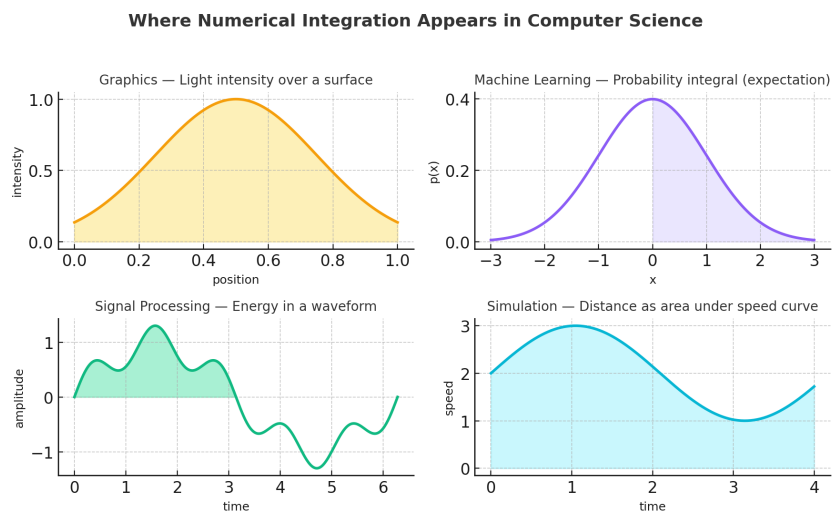
As we move from rectangles to trapezoids to parabolas, the approximation becomes smoother and the estimated area (distance) becomes more accurate.

1.2 Why do we need this in Computer Science?

Even though integrals come from mathematics, they appear everywhere in computing:

- In **graphics**, to compute average colors, light energy, or blur.
- In **machine learning**, for expectations over probability distributions.
- In **signal processing**, for converting between time and frequency domains.
- In **simulation**, to estimate energy, work, or total quantities over time.

Whenever a computer handles continuous phenomena, integration becomes necessary, but it must be done *numerically*.



1.3 Analytic vs. numeric integration

An **analytic** integral uses symbolic formulas:

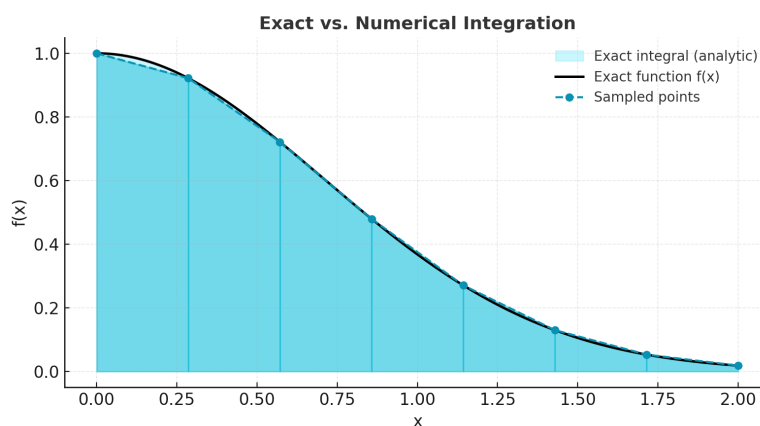
$$\int x^2 dx = \frac{x^3}{3} + C.$$

This is exact, but possible only for simple functions.

A **numeric** integral instead uses computation:

$$\int_0^1 e^{-x^2} dx \approx 0.7468.$$

Here, no formula exists. We approximate using quadrature rules.

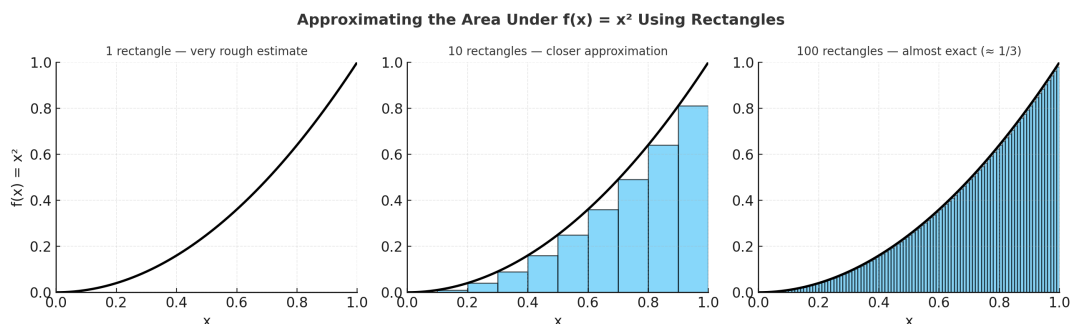


1.4 Example: measuring the area under a curve

Suppose we want to find the area under $f(x) = x^2$ between $x = 0$ and $x = 1$. We can draw the curve and fill the area under it with rectangles:

- Using **1 rectangle**, we get a rough estimate.
- Using **10 rectangles**, the estimate gets much closer.
- Using **100 rectangles**, the result almost matches the true area $\frac{1}{3}$.

The more subintervals we use, the better the approximation. Later, we will describe this formally using the concept of **order of convergence**.



2 Fundamental Concepts

Numerical integration, or *quadrature*, aims to approximate the value of a definite integral

$$I = \int_a^b f(x) dx$$

using only a finite number of evaluations of the function $f(x)$ at selected points.

2.1 General idea

If $f(x)$ is smooth but not easy to integrate analytically, we replace it with a simpler function, often a low-degree polynomial, whose integral can be computed exactly.

$$\int_a^b f(x) dx \approx Q_n(f) = \sum_{i=1}^n \omega_i f(c_i)$$

Here:

- c_i are the **nodes** (the points where f is evaluated);
- ω_i are the **weights**, which depend on the interval $[a, b]$ and the chosen rule.

The approximation $Q_n(f)$ is called a **quadrature formula** with n nodes.

Note

Intuitively, each term $\omega_i f(c_i)$ represents a small “area” contribution. The sum of all these weighted contributions approximates the total area under $f(x)$.

2.2 Error and convergence

The accuracy of a quadrature formula is measured by the error

$$E(n) = \left| \int_a^b f(x) dx - Q_n(f) \right|.$$

The error typically decreases as we increase the number of points n (that is, as we refine the partition).

We say that a quadrature formula has **algebraic convergence of order p** if the error behaves like

$$E(n) = O\left(\frac{1}{n^p}\right).$$

This means that doubling the number of subintervals reduces the error approximately by a factor of 2^p . The constant p is called the **order of accuracy** of the method.

Note

For example, the composite trapezoidal rule has $p = 2$, while the composite Simpson rule has $p = 4$. These values reflect how quickly the global error decreases as the step size $h = (b - a)/n$ becomes smaller.

2.3 Polynomial interpolation and the Lagrange form

A central idea behind numerical integration is to replace a complicated function $f(x)$ with a simpler one that is easy to integrate exactly. One of the most powerful ways to do this is to use **polynomial interpolation**.

Motivation. Suppose we know the values of $f(x)$ only at n points $c_1, c_2, \dots, c_n$. We would like to construct a polynomial $P_{n-1}(x)$ of degree at most $n - 1$ that passes exactly through these points:

$$P_{n-1}(c_i) = f(c_i), \quad i = 1, 2, \dots, n.$$

This polynomial can then be integrated analytically, providing a good approximation to the true integral of $f(x)$.

Lagrange interpolation formula. The polynomial that satisfies these conditions can be written as a linear combination of special basis polynomials called the **Lagrange basis functions**:

$$f(x) \approx P_{n-1}(x) = \sum_{i=1}^n f(c_i) \ell_i(x),$$

where each $\ell_i(x)$ is defined by

$$\ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - c_j}{c_i - c_j}.$$

Each $\ell_i(x)$ is constructed with respect to the fixed interpolation nodes $c_1, \dots, c_n$. Here, x is a continuous variable in $[a, b]$.

By construction,

$$\ell_i(c_j) = \delta_{ij} \quad \text{for } i, j = 1, \dots, n.$$

Moreover, the Lagrange basis forms a *partition of unity*:

$$\sum_{i=1}^n \ell_i(x) = 1 \quad \text{for all } x \in [a, b].$$

Proof sketch. Let $S(x) = \sum_{i=1}^n \ell_i(x)$. Then S is a polynomial of degree $\leq n - 1$ and satisfies $S(c_j) = \sum_i \ell_i(c_j) = 1$ for $j = 1, \dots, n$. Since two polynomials of degree $\leq n - 1$ that agree at n distinct points are identical, we conclude $S(x) \equiv 1$ on $[a, b]$.

Note

Geometrically, $\ell_i(x)$ is the unique polynomial that equals 1 at node c_i and 0 at all other nodes. This makes $\ell_i(x)$ act as a “selector” for $f(c_i)$, ensuring the interpolant passes through all known points.

From interpolation to integration. If we integrate the interpolating polynomial instead of the exact function, we obtain a general quadrature formula:

$$Q_n(f) = \int_a^b P_{n-1}(x) dx = \sum_{i=1}^n f(c_i) \underbrace{\int_a^b \ell_i(x) dx}_{\omega_i}.$$

The integrals of the basis polynomials define the **weights** ω_i , which determine how much each sample $f(c_i)$ contributes to the total integral.

Note

This perspective explains the origin of all classical numerical integration formulas:

- The **Midpoint rule** uses one interpolation node (a constant polynomial);
- The **Trapezoidal rule** uses two nodes and a linear polynomial;
- The **Simpson rule** uses three nodes and a quadratic polynomial.

All of them are special cases of the general Lagrange-based quadrature framework.

Example: three equally spaced nodes. For $n = 3$ and $[a, b] = [0, 1]$, the nodes are typically chosen as

$$c_1 = 0, \quad c_2 = \frac{1}{2}, \quad c_3 = 1.$$

The corresponding Lagrange polynomials are:

$$\ell_1(x) = 2(x - \frac{1}{2})(x - 1), \quad \ell_2(x) = -4x(x - 1), \quad \ell_3(x) = 2x(x - \frac{1}{2}).$$

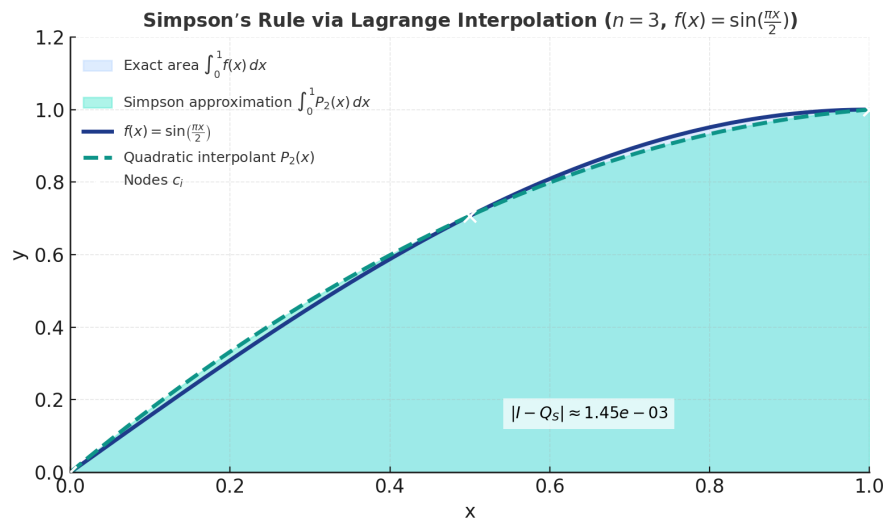
Integrating them gives the weights:

$$\omega_1 = \frac{1}{6}, \quad \omega_2 = \frac{4}{6}, \quad \omega_3 = \frac{1}{6}.$$

Hence, the formula becomes

$$\int_0^1 f(x) dx \approx \frac{1}{6} [f(0) + 4f(\frac{1}{2}) + f(1)],$$

which is precisely the **Simpson rule**.



2.4 Accuracy and polynomial degree

If the quadrature formula integrates exactly all polynomials up to degree m , we say it has **precision degree** m (or “order $m + 1$ ” in some conventions). In general:

$$Q_n(p) = \int_a^b p(x) dx \quad \text{for all polynomials } p(x) \text{ with } \deg(p) \leq m.$$

A higher degree of exactness implies smaller error for smooth functions, since smooth functions can be locally approximated by polynomials.

2.5 Divide and conquer: composite formulas

Instead of using one single large polynomial across $[a, b]$, we can split the interval into smaller subintervals of size h and apply a simple rule (e.g. midpoint, trapezoidal, or Simpson) on each piece. This leads to a **composite rule**:

$$Q_h(f) = \sum_{k=0}^{m-1} Q(f; [x_k, x_{k+1}]).$$

This “divide and conquer” strategy helps control local errors, leading to an overall global error that decreases as $O(h^p)$, where p is the order of the local rule.

Note

Composite quadrature formulas are the practical choice for computers: they reuse simple rules but apply them many times with small step size h .

3 Formulas with Equally Spaced Nodes

In many practical situations, the function $f(x)$ is sampled at equally spaced points. This leads to a family of simple yet powerful quadrature formulas known as the **midpoint**, **trapezoidal**, and **Simpson** rules.

These methods are all derived from integrating a polynomial interpolant passing through 1, 2, or 3 equally spaced nodes.

3.1 Midpoint Rule and its Derivation from Lagrange Interpolation

The **midpoint rule** (also known as the *rectangle rule*) is the simplest quadrature formula that uses a single sample point. It approximates the area under $f(x)$ on $[a, b]$ by evaluating the function at the midpoint:

$$Q_M(f; a, b) = (b - a) f\left(\frac{a + b}{2}\right).$$

Derivation from Lagrange interpolation. We start by constructing the Lagrange interpolant of degree 0 (a constant polynomial) through one node

$$c_1 = \frac{a + b}{2}.$$

The corresponding Lagrange basis polynomial is simply

$$\ell_1(x) = 1.$$

Thus, the interpolant is

$$P_0(x) = f(c_1) \ell_1(x) = f(c_1),$$

a constant function equal to f evaluated at the midpoint.

Integrating this polynomial over $[a, b]$ gives the quadrature formula:

$$Q_M(f; a, b) = \int_a^b P_0(x) dx = f(c_1) \int_a^b 1 dx = (b - a) f\left(\frac{a + b}{2}\right).$$

Hence, the **weight** associated to this single node is

$$\omega_1 = \int_a^b \ell_1(x) dx = (b - a).$$

Accuracy and interpretation. The midpoint rule integrates all constant and linear functions exactly. For a general smooth function f , the only part it cannot capture is the *curvature* — how much f bends between a and b . This curvature is measured by the second derivative $f''(x)$.

If we think of $f(x)$ as locally a small parabola

$$f(x) \approx (\text{a straight line}) + \frac{1}{2} f''(\xi)(x - c_1)^2,$$

then the linear part is integrated exactly, while the quadratic term produces a small remaining

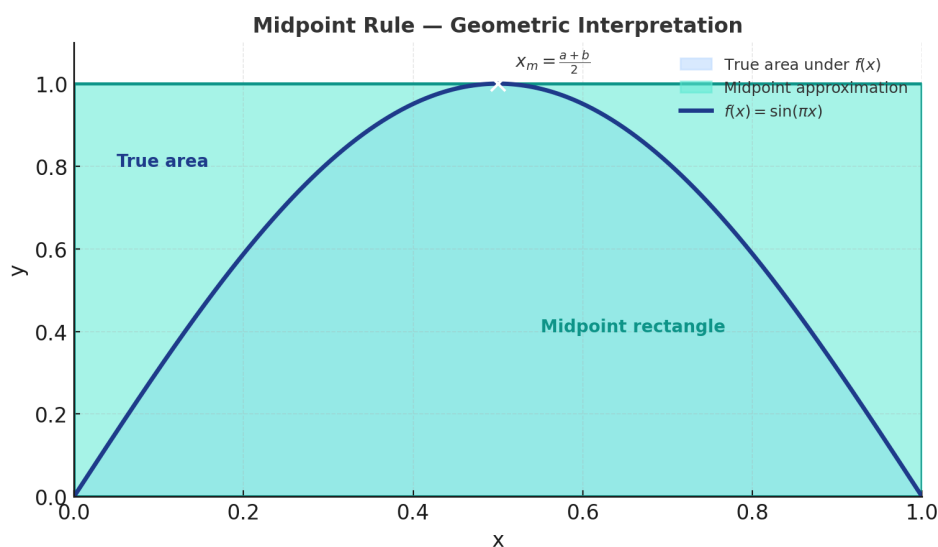
area.

Because this extra term involves $(x - c_1)^2$, the error scales like the square of the interval width:

$$E_M(f) \propto (b - a)^2.$$

In other words, halving the interval roughly divides the error by four, a signature of **second-order accuracy**.

Geometric interpretation. Geometrically, we replace the possibly curved graph of $f(x)$ by a rectangle of height $f((a + b)/2)$ and base length $(b - a)$.



Geometric interpretation of the midpoint rule: the curved area under $f(x) = \sin(\pi x)$ is replaced by a rectangle of height $f((a + b)/2)$ and width $(b - a)$.

Note

If instead of the midpoint we evaluate f at one of the endpoints a or b , we obtain the **left** or **right rectangle rule**:

$$Q_R(f; a, b) = (b - a)f(a) \quad \text{or} \quad (b - a)f(b).$$

These formulas are still based on a constant approximation, but they are *not symmetric*, which makes them only **first-order accurate** ($E = O(h)$).

In contrast, the midpoint rule is symmetric with respect to the interval, so the leading first-order errors on the left and right cancel out, raising the accuracy to **second order**.

3.2 Trapezoidal Rule and its Derivation from Lagrange Interpolation

The **trapezoidal rule** approximates the integral of $f(x)$ over $[a, b]$ by replacing f with the straight line that passes through the points $(a, f(a))$ and $(b, f(b))$:

$$Q_T(f; a, b) = \frac{b-a}{2} [f(a) + f(b)].$$

Derivation from Lagrange interpolation. We construct the Lagrange interpolating polynomial of degree 1 through two equally spaced nodes:

$$c_1 = a, \quad c_2 = b.$$

The corresponding Lagrange basis polynomials are

$$\ell_1(x) = \frac{b-x}{b-a}, \quad \ell_2(x) = \frac{x-a}{b-a}.$$

Hence, the linear interpolant is

$$P_1(x) = f(a) \ell_1(x) + f(b) \ell_2(x) = f(a) \frac{b-x}{b-a} + f(b) \frac{x-a}{b-a}.$$

Integrating this polynomial over $[a, b]$ gives:

$$Q_T(f; a, b) = \int_a^b P_1(x) dx = f(a) \int_a^b \ell_1(x) dx + f(b) \int_a^b \ell_2(x) dx.$$

Since

$$\int_a^b \ell_1(x) dx = \int_a^b \frac{b-x}{b-a} dx = \frac{b-a}{2}, \quad \int_a^b \ell_2(x) dx = \int_a^b \frac{x-a}{b-a} dx = \frac{b-a}{2},$$

the formula becomes

$$Q_T(f; a, b) = \frac{b-a}{2} [f(a) + f(b)].$$

Thus, the **weights** are equal:

$$\omega_1 = \omega_2 = \frac{b-a}{2}.$$

Accuracy and interpretation. The trapezoidal rule integrates all straight lines exactly, since the area under a line between a and b is a perfect trapezoid. For curved functions, the only part that the rule misses is the *curvature* of f , measured by the second derivative $f''(x)$.

If $f(x)$ bends upward or downward, the straight segment connecting $f(a)$ and $f(b)$ slightly over- or underestimates the real curve. This bending effect grows with the width of the interval squared, $(b-a)^2$ — meaning that when we halve the interval, the error becomes about four times smaller:

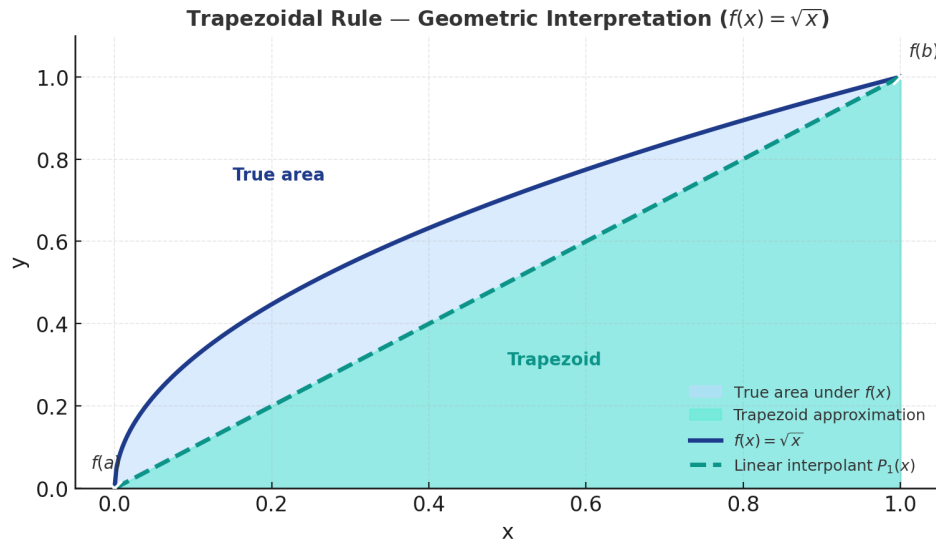
$$E_T(f) \propto (b-a)^2.$$

Hence, the trapezoidal rule is said to be **second-order accurate**.

Note

Both the midpoint and trapezoidal rules share this $O(h^2)$ accuracy because they are symmetric — they balance the errors on the left and right sides of each interval.

Geometric interpretation. Geometrically, the trapezoidal rule replaces the curved area under $f(x)$ by a single trapezoid with vertices at $(a, 0)$, $(a, f(a))$, $(b, f(b))$, and $(b, 0)$.



Geometric interpretation of the trapezoidal rule for $f(x) = \sqrt{x}$. The linear interpolant $P_1(x)$ (turquoise dashed) connects $(a, f(a))$ and $(b, f(b))$, forming a trapezoid that slightly overestimates the true area under f .

3.3 Simpson's Rule and its Derivation from Lagrange Interpolation

Simpson's rule is obtained by interpolating $f(x)$ with a **quadratic** (degree-2) polynomial through three equally spaced nodes:

$$c_1 = a, \quad c_2 = \frac{a+b}{2}, \quad c_3 = b.$$

The Lagrange basis polynomials are:

$$\begin{aligned} \ell_1(x) &= \frac{(x - \frac{a+b}{2})(x - b)}{(a - \frac{a+b}{2})(a - b)}, \\ \ell_2(x) &= \frac{(x - a)(x - b)}{(\frac{a+b}{2} - a)(\frac{a+b}{2} - b)}, \\ \ell_3(x) &= \frac{(x - a)(x - \frac{a+b}{2})}{(b - a)(b - \frac{a+b}{2})}. \end{aligned}$$

The quadratic interpolant is

$$P_2(x) = f(a) \ell_1(x) + f\left(\frac{a+b}{2}\right) \ell_2(x) + f(b) \ell_3(x).$$

Integrating $P_2(x)$ over $[a, b]$ gives

$$Q_S(f; a, b) = \int_a^b P_2(x) dx = f(a) \int_a^b \ell_1(x) dx + f\left(\frac{a+b}{2}\right) \int_a^b \ell_2(x) dx + f(b) \int_a^b \ell_3(x) dx.$$

Evaluating these integrals yields the weights

$$\omega_1 = \frac{b-a}{6}, \quad \omega_2 = \frac{4(b-a)}{6}, \quad \omega_3 = \frac{b-a}{6},$$

so that

$$Q_S(f; a, b) = \frac{b-a}{6} [f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)].$$

Why the accuracy is fourth order. The quadratic interpolant exactly reproduces all polynomials up to degree 2, and even integrates *cubic* functions exactly. This means the first term that produces an error is the quartic one.

If we expand $f(x)$ locally as a Taylor series around the midpoint,

$$f(x) = f(m) + f'(m)(x-m) + \frac{f''(m)}{2}(x-m)^2 + \frac{f^{(3)}(m)}{6}(x-m)^3 + \dots,$$

then both the midpoint and trapezoidal rules already cancel the terms up to $(x-m)^2$. Simpson's rule combines them in such a way that even the $(x-m)^3$ term cancels, leaving the first non-zero error term proportional to $(b-a)^4 f^{(4)}(\xi)$.

Hence, the global error behaves as

$$E_S(f) = O(h^4),$$

which makes Simpson's rule **fourth-order accurate**: halving the step size reduces the error by roughly a factor of 16.

Note

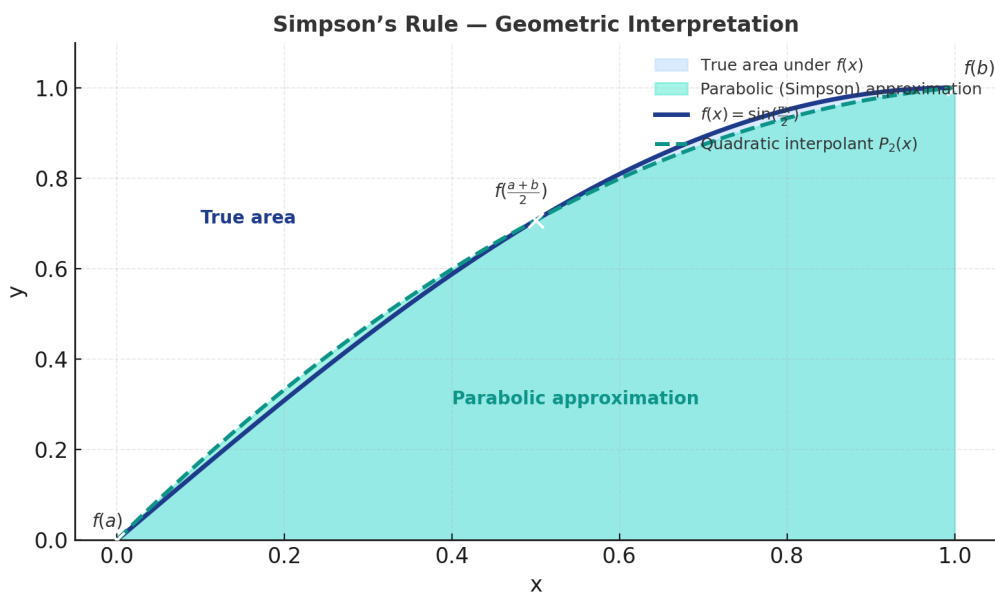
[Why cubic terms vanish] A key reason for Simpson's high accuracy is **symmetry**. The three nodes a , $\frac{a+b}{2}$, and b are equally spaced around the midpoint. When we expand $f(x)$ as a Taylor series around the midpoint $m = \frac{a+b}{2}$,

$$f(x) = f(m) + f'(m)(x-m) + \frac{f''(m)}{2}(x-m)^2 + \frac{f^{(3)}(m)}{6}(x-m)^3 + \dots,$$

the odd-power terms, such as $(x-m)$ and $(x-m)^3$, are *antisymmetric*: they take opposite signs on the left and right halves of the interval.

Because Simpson's rule is symmetric, these opposite contributions cancel out exactly when integrated. As a result, even the cubic term produces zero error, and the first nonzero error term involves $(x-m)^4$.

This cancellation explains why Simpson's rule achieves **fourth-order accuracy** despite using only three points.



Simpson's rule as degree-2 Lagrange interpolation: the parabolic interpolant (dashed) passes through $(a, f(a))$, $(\frac{a+b}{2}, f(\frac{a+b}{2}))$, and $(b, f(b))$, providing a highly accurate area approximation.

3.4 Composite Formulas

So far, each rule (midpoint, trapezoidal, Simpson) was applied on a single interval $[a, b]$. In practice, this gives only a rough estimate if f varies significantly. To improve accuracy, we divide the interval into n smaller, equally spaced pieces:

$$a = x_0 < x_1 < \dots < x_n = b, \quad h = \frac{b-a}{n}.$$

We then apply the same local rule on each small piece $[x_k, x_{k+1}]$ and sum all contributions:

$$Q_h(f) = \sum_{k=0}^{n-1} Q(f; x_k, x_{k+1}).$$

This produces the **composite** rules:

- **Composite Midpoint Rule:**

$$Q_M^{(n)}(f) = h \sum_{k=0}^{n-1} f\left(\frac{x_k + x_{k+1}}{2}\right).$$

- **Composite Trapezoidal Rule:**

$$Q_T^{(n)}(f) = \frac{h}{2} [f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b)].$$

- **Composite Simpson Rule:**

$$Q_S^{(n)}(f) = \frac{h}{6} [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n)],$$

where n must be even.

Error behavior. Each local subinterval contributes a small local error $E_{\text{local}} = O(h^{p+1})$. Summing over $n = (b - a)/h$ intervals yields a global error

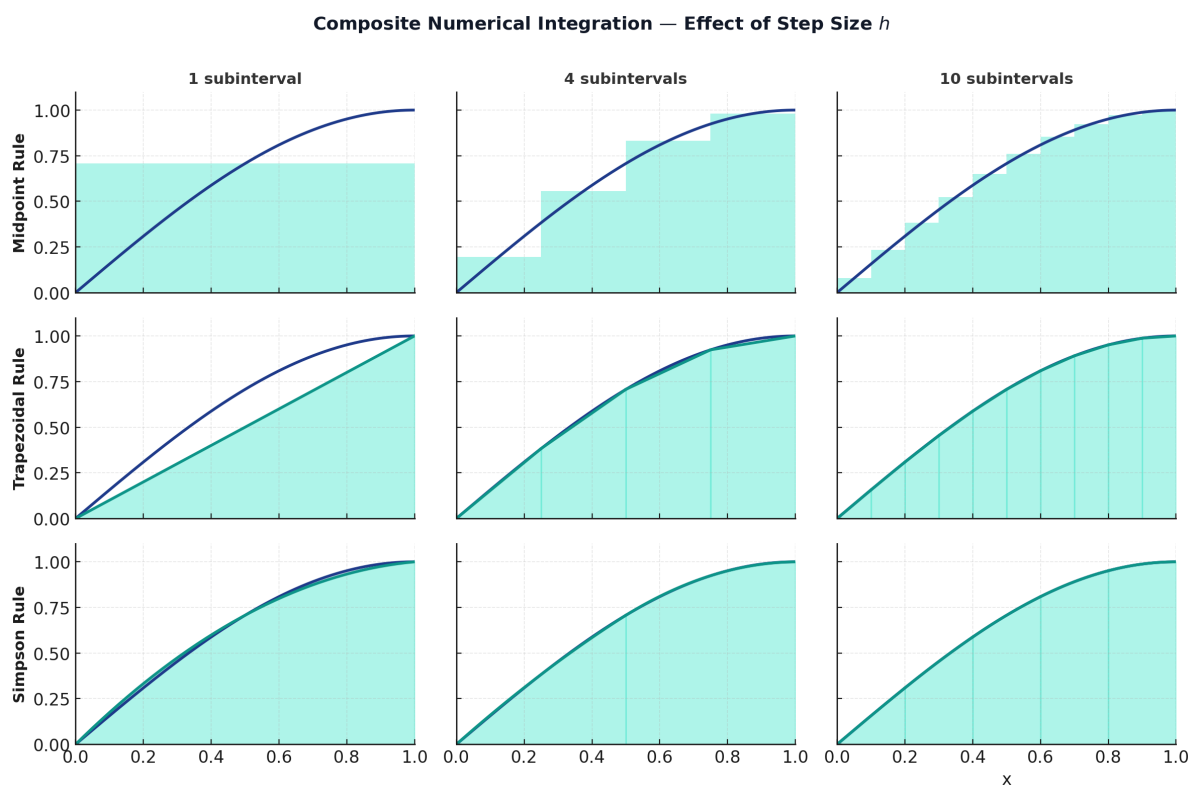
$$E_{\text{global}} = O(h^p),$$

where p is the order of accuracy of the base rule:

$$p = \begin{cases} 2, & \text{for midpoint and trapezoid,} \\ 4, & \text{for Simpson.} \end{cases}$$

Note

Composite formulas are the ones actually used in practice and in code. They combine simplicity with control over accuracy: by halving h , we can predict how much the global error will decrease. This “divide and conquer” idea is the foundation of modern numerical integration.



Comparison of composite numerical integration methods for different step sizes h . Each row shows a

different rule (Midpoint, Trapezoidal, Simpson), while columns correspond to 1, 4, and 10 subintervals. Smaller h yields visibly more accurate approximations.

Summary: Local vs. Global Accuracy

The following table compares the local and global error behavior for the three main rules:

Rule	Local error	Global error	Global order p
Midpoint	$O(h^3)$	$O(h^2)$	2
Trapezoidal	$O(h^3)$	$O(h^2)$	2
Simpson	$O(h^5)$	$O(h^4)$	4

Note

[Clarification: what we actually computed] When we previously said that the midpoint and trapezoidal rules have order $p = 2$, and Simpson's rule has order $p = 4$, we were implicitly referring to the **global accuracy**. That is, the behavior of the total error when the interval $[a, b]$ is divided into many smaller subintervals of length h .

The theoretical derivation of $E_{\text{local}} = O(h^{p+1})$ on a single interval leads directly to

$$E_{\text{global}} = n E_{\text{local}} = O(h^p),$$

since $n = (b - a)/h$ subintervals are used. Therefore, all the “orders” p we quoted for each rule correspond to this global convergence rate.

4 Bonus: Monte Carlo Integration

When the function $f(x)$ is complicated or high-dimensional, deterministic quadrature rules become impractical. Instead, we can use a probabilistic approach based on **random sampling**.

$$I = \int_a^b f(x) dx \approx (b-a) \frac{1}{N} \sum_{i=1}^N f(x_i),$$

where each x_i is drawn uniformly at random from $[a, b]$.

Intuition. Each $f(x_i)$ acts like a random “probe” of the function. The average of many such probes estimates the mean value of f on $[a, b]$. As N increases, the estimate converges to the true value.

Connection with expected value. If X is a random variable uniformly distributed on $[a, b]$, its probability density is $p(x) = \frac{1}{b-a}$. Then the expected value of $f(X)$ is

$$\mathbb{E}[f(X)] = \int_a^b f(x) p(x) dx = \frac{1}{b-a} \int_a^b f(x) dx.$$

Rearranging gives

$$\int_a^b f(x) dx = (b-a) \mathbb{E}[f(X)].$$

Thus, by sampling independent copies $X_1, \dots, X_N$, we approximate the expectation by

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{i=1}^N f(X_i),$$

and hence the integral by

$$I_N = (b-a) \frac{1}{N} \sum_{i=1}^N f(X_i).$$

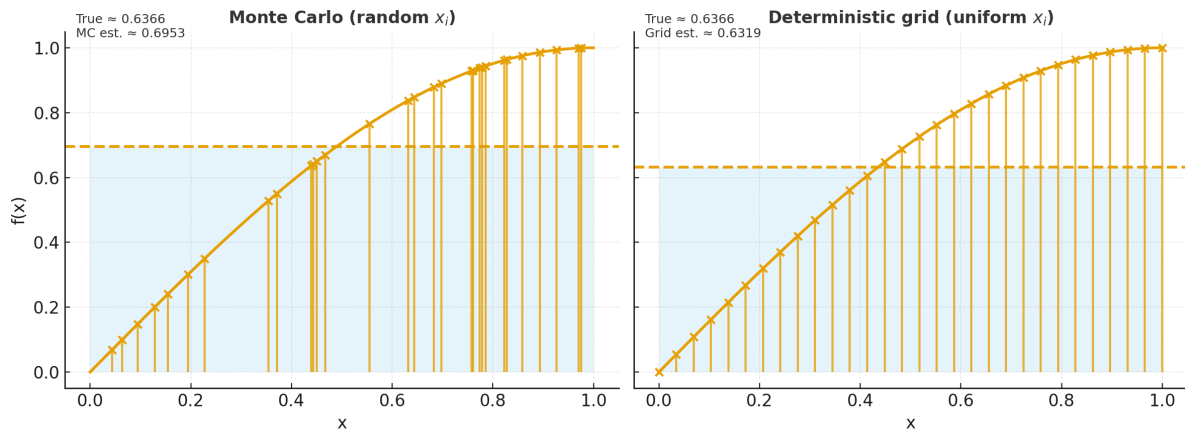
Convergence as N grows. By the **Law of Large Numbers**, $I_N \rightarrow I$ as $N \rightarrow \infty$. By the **Central Limit Theorem**, the fluctuations around the true value decay like

$$E_N = O\left(\frac{1}{\sqrt{N}}\right).$$

This convergence is slow but dimension-independent, which makes Monte Carlo ideal for high-dimensional integrals.

Note

Monte Carlo integration trades deterministic precision for statistical simplicity: it works even for irregular, noisy, or multidimensional functions.



Left: Monte Carlo (random samples x_i) — the estimate is $(b-a)\frac{1}{N}\sum f(x_i)$. Right: deterministic uniform grid (not Monte Carlo). Monte Carlo relies on i.i.d. random sampling and converges as $O(1/\sqrt{N})$ by the LLN/CLT.

5 Numerical Integration in Python

We now connect the theoretical rules with their practical implementation in Python. All methods below are written using only NumPy, and we also compare them with the built-in routines from `scipy.integrate`.

Note

These examples illustrate how the formulas for midpoint, trapezoidal, and Simpson's rules translate directly into working Python code. By changing the number of subintervals, students can observe the convergence behavior.

5.1 Setup

```
import numpy as np
from scipy.integrate import quad, trapezoid, simpson

# Function to integrate
def f(x):
    return np.exp(-x**2)

# Integration bounds
a, b = 0, 1
```

5.2 Manual Implementations

Each rule is implemented manually so we can understand its structure and order of accuracy.

```
# =====
# (a) Midpoint Rule
# =====
def midpoint_rule(f, a, b, n):
    """Composite Midpoint Rule with n subintervals"""
    h = (b - a) / n
    mids = a + (np.arange(n) + 0.5) * h
    return h * np.sum(f(mids))

# =====
# (b) Trapezoidal Rule
# =====
def trapezoidal_rule(f, a, b, n):
    """Composite Trapezoidal Rule"""
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    return (h / 2) * (f(x[0]) + 2 * np.sum(f(x[1:-1])) + f(x[-1]))
```

```
# =====
# (c) Simpson's Rule
# =====
def simpson_rule(f, a, b, n):
    """Composite Simpson's Rule (requires n even)"""
    if n % 2 != 0:
        n += 1
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    return (h / 3) * (f(x[0]) + 4 * np.sum(f(x[1:-1:2])) +
                    2 * np.sum(f(x[2:-2:2])) + f(x[-1]))
```

5.3 Evaluate and Compare

```
# Number of subintervals
n = 10

I_mid = midpoint_rule(f, a, b, n)
I_trap = trapezoidal_rule(f, a, b, n)
I_simp = simpson_rule(f, a, b, n)

print("Midpoint rule:  ", I_mid)
print("Trapezoidal rule:", I_trap)
print("Simpson rule:    ", I_simp)
```

5.4 Using scipy.integrate

The `scipy.integrate` module provides high-accuracy numerical integration routines. We can use it to obtain a reference value and to apply the same rules on uniform grids.

```
# =====
# 1. Adaptive high-precision integration (reference value)
# =====
I_true, err = quad(f, a, b)
print("True value (quad):", I_true)

# =====
# 2. Uniform-grid integration (fixed subintervals)
# =====
x = np.linspace(a, b, 101)
I_trap_scipy = trapezoid(f(x), x)
I_simp_scipy = simpson(f(x), x)

print("Trapezoidal (SciPy):", I_trap_scipy)
print("Simpson (SciPy):    ", I_simp_scipy)
```

5.5 Typical Output Example

```
True value (quad): 0.746824...  
Midpoint rule:    0.746855  
Trapezoidal rule: 0.746462  
Simpson rule:     0.746824  
Trapezoidal (SciPy): 0.746467  
Simpson (SciPy):  0.746823
```